Folios — AI Governance

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-06-03

This document describes how Folios uses AI (Pip) responsibly. It covers what Pip can and can't do, what guardrails are in place, how cost and abuse are controlled, and how the user stays in command.

For the technical AI architecture, see architecture.md. For what data crosses to Anthropic, see data-handling.md.

Stance

Folios takes AI seriously. Pip is built on three principles:

1. **The user always reviews before AI writes to their data.** Pip can *propose*; it cannot silently *execute*. Every action of consequence (creating items, updating projects, closing tasks) passes through a human-approved preview.
2. **The user can always see what Pip is doing.** Every Pip API call is logged with mode, tokens, and timestamp. Every learning data point Pip captures is visible and editable.
3. **The user can always disagree.** Override mechanisms exist for every Pip decision. Disagreements feed the learning loop so Pip improves; they don't just get ignored.

What Pip can do

Capability	Where it fires
Summarize meeting notes into a structured plan	End & Summarize on a meeting
Generate pre-call briefs	"Brief Me" button on Cadence Hub
Answer questions about user's accounts	Pip view (chat)
Extract action items from short notes	Quick Touchpoint modal
Suggest task assignments based on learned patterns	Inside summarize plans
Route tasks across accounts based on user notes	Inside summarize plans
Distill correction history into account lessons	Background compression pass
Execute confirmed tool calls (add item, set follow-up, close item, etc.)	User-approved actions from chat

What Pip cannot do

Capability	Why not
Send emails on the user's behalf	Not built; no email integration
Modify accounts/items without user preview	Design rule — every consequential write is human-approved
Access data outside the user's own (or org-mates') scope	Enforced by Supabase RLS at the database layer
Persist anything to other users' data	Same — RLS enforces user/org scope
Speak for the user externally (Slack, email, etc.)	No external write integrations exist
Auto-trigger on a schedule	No cron jobs; Pip only runs when the user invokes a Pip surface
Override user corrections	Corrections are immutable; Pip reads them as instructions

Where the user is always in control

Summarize-preview gate

After Pip generates a meeting plan, the **PipSummarizePreview** modal opens. Nothing is written to the database until the user clicks Apply. The user can:

- Uncheck any row to decline it
- Edit the row title before applying
- Reassign tasks to different people
- Reroute tasks to different accounts
- Expand "see source" to verify which slice of notes triggered each row
- Add missed items Pip didn't catch

- Cancel everything (preserves the meeting summary, applies no changes)

Chat tool-call confirmation

When Pip in chat mode wants to write to the database, it returns a tool call. Tool calls are classified:

- **Frictionless** (e.g., adding a quick task the user explicitly asked for) — auto-execute.
- **Confirm-required** (e.g., closing a tracked item, setting a follow-up date) — surfaced as a **PipActionCard** that the user must explicitly approve.

The threshold lives in `src/lib/pipTools.js` (`CONFIRM_THRESHOLD`) and is biased toward asking for confirmation.

Glossary and assignment hints

The user can edit or delete any glossary entry (Settings → Pip Glossary). The user can override any assignment hint by reassigning in the preview modal; the override re-trains Pip.

Lessons learned

The distilled `pip_account_state.lessons_learned` paragraph is read by Pip but is currently a developer-managed value. A user-facing edit surface is planned.

Cancel & override everywhere

- Summarize plan cancel preserves the summary, applies nothing.
- Brief Me can be re-generated.
- Quick task suggestions can be unchecked.
- Pip's account routing can be overridden in preview AND post-apply (via the TaskDetailPanel account override).

Prompt-injection guards

User-supplied text (meeting notes, account context, glossary entries) flows into Pip's prompts. Without guards, malicious or accidental input could try to override Pip's instructions.

Mitigations in place:

1. **Sandwich structure.** System instructions wrap the user content on both sides. Pip is told explicitly to ignore instructions embedded in user data.
2. **Scope cues.** The prompt frames user-supplied data with explicit labels ("USER NOTES BEGIN" / "USER NOTES END") so Pip doesn't mistake user text for system instructions.
3. **Strict output schema.** Summarize mode requires JSON output matching a specific schema. Off-schema output is rejected.
4. **No autonomous action.** Even if Pip is tricked into proposing bad actions, the preview gate prevents anything from being written without user approval.
5. **Output rendering.** Pip's text output is rendered through a markdown component that strips arbitrary HTML. No `eval`, no `dangerouslySetInnerHTML`, no script execution surface.

These guards are defense in depth. A prompt-injection attack would need to (a) get the model to produce bad output AND (b) get the human reviewing the preview to approve it AND (c) bypass the rendering safety. All three layers are non-trivial to defeat.

Cost controls

AI inference is metered, and runaway cost is a real risk. Folios's defenses:

Rate limiting

- **20 Pip API calls per minute per user**, enforced in `/api/pip` via in-memory counter.

- Prevents both intentional abuse and accidental client loops.

Model selection

- **claude-haiku-4-5-20251001** is the default for all Pip operations.
- **Sonnet** is reserved for high-synthesis paths (Brief Me).
- No Opus calls in production code.

Prompt caching

- **4 cache breakpoints** in summary mode (system / glossary / roster / items+tasks).
- Multi-call sessions get ~70% cache hit rate; cached tokens are ~90% cheaper than uncached.

Trivial-draft short-circuit

- Drafts under 100 characters never call the API at all.
- The summarize modal handles the empty-plan path locally with a clear "nothing to summarize here" message.

Pre-computed answers

- `classifyIntent` routes deterministic questions ("how many open items do I have?") to local computation, no API call.
- Common stats are computed from already-loaded data.

Saved outputs

- `pip_summary` and `pip_email` columns on `folio_meetings` cache generated content. Once generated, never regenerated.

Telemetry

- Every API call logs to `folio_pip_usage`: tokens in, tokens out, cache tokens, mode, timestamp.
- Per-user dashboard in Settings → Pip Usage shows daily/weekly cost.
- Visible cost is the most effective behavior modifier.

Hallucination & accuracy posture

Pip is an LLM. It can be wrong. Folios mitigates this with:

Grounding

- Every Pip call is grounded in the user's actual data (accounts, items, contacts, glossary, lessons learned). Pip is not asked to recall general knowledge.
- **Context parity:** Both chat-Pip (Pip view) and summarize-Pip (End & Summarize) receive the same glossary entries and per-account lessons learned, so Pip's vocabulary and learned patterns are consistent across both surfaces.
- The user's notes are always in context; Pip is summarizing text the user wrote, not generating from thin air.

Verification surface

- The "see source" expander on every plan row shows the slice of the user's notes that triggered the proposed action. Users can verify Pip's reasoning before approving.

Correction loop

- When Pip is wrong, the user corrects it. Corrections become training signal for future calls (`pip_correction_log`).

- After a few weeks of use, repeat mistakes become rare because the correction history is read back into every call.

No fabrication encouragement

- Prompts explicitly instruct Pip to skip rather than invent. If Pip doesn't see a clear action item, the plan returns "skip" rows, not made-up tasks.
- Empty plans are a valid output. Pip is told not to pad.

Confidence signals

- Plan rows include a confidence flag. Low-confidence rows get a yellow dot in the preview UI so users review them more carefully.

Anthropic-side governance

Folios uses the standard Anthropic API. Key terms:

- **No training on customer inputs.** Anthropic does not train models on API customer inputs by default.
- **30-day input/output retention** for abuse monitoring, then deletion. Governed by Anthropic's standard API terms.
- **SOC 2 Type 2 attested.**
- **US-based inference** under current configuration.

Anthropic's full data-handling and trust posture is at trust.anthropic.com.

Audit & transparency

What's logged

- Every Pip API call: `folio_pip_usage` (mode, tokens, timestamp)
- Every write Pip triggers (via tool call or summarize Apply): `folio_activity` (action, resource, timestamp, user)
- Every correction the user makes to Pip output: `pip_correction_log`

What the user can see

- **Settings → Pip Usage** — cost dashboard
- **Settings → Activity** — write audit log
- **Settings → Pip Glossary** — every learned term, editable
- **Settings → Diagnostics** — every error including Pip errors

What the user can NOT see today

- `pip_account_state.lessons_learned` — distilled paragraph per account. Read by Pip, currently not surfaced for user inspection. Planned: surface in Settings → Pip Memory.
- `pip_assignment_hints` — Pip's learned routing patterns. Planned: surface and make editable.

Failure modes & contingencies

Anthropic outage

- Pip features fail gracefully. Buttons show "Pip is offline" state.

- Non-Pip features (account CRUD, meeting capture, item management) continue working normally.
- Saved Pip outputs (summary, brief) remain readable from the DB.

Rate-limit exceeded

- `/api/pip` returns 429.
- Client surfaces a clear message ("Slow down — Pip needs a moment") and disables the trigger until cooldown.

Unexpected output

- JSON schema validation on summarize output. Malformed responses fall back to a synthesized empty plan with an error log entry.
- Streaming failures captured with full stack to `folio_errors`.

Model behavior change

- Folios is pinned to a specific model version (`claude-haiku-4-5-20251001`). Anthropic's model versioning guarantees stable behavior for that version.
- Model upgrades are explicit code changes, never automatic.

Future hardening

These are not gaps — they're planned improvements:

- **User-facing memory editing**: lessons_learned and assignment hints surfaced in Settings for inspection and override.
- **Hard opt-out toggle**: Settings switch to fully disable Pip per-session (already possible by avoiding Pip surfaces; explicit toggle would simplify).

- **Daily cost cap per user**: configurable hard ceiling beyond the per-minute rate limit.
- **Anonymized prompt sampling for QA**: spot-check Pip's outputs for quality regression (with user consent).
- **Periodic prompt review**: quarterly audit of Pip's system prompts as model and feature surface evolve.

Contact

For AI governance questions: chris.vasconcellos97@gmail.com